LAPORAN PRAKTIKUM JOBSHEET 5

Disusun Oleh:

HAFIZH ARRASYIID SYAHBANA

254107060026

SIB 1A

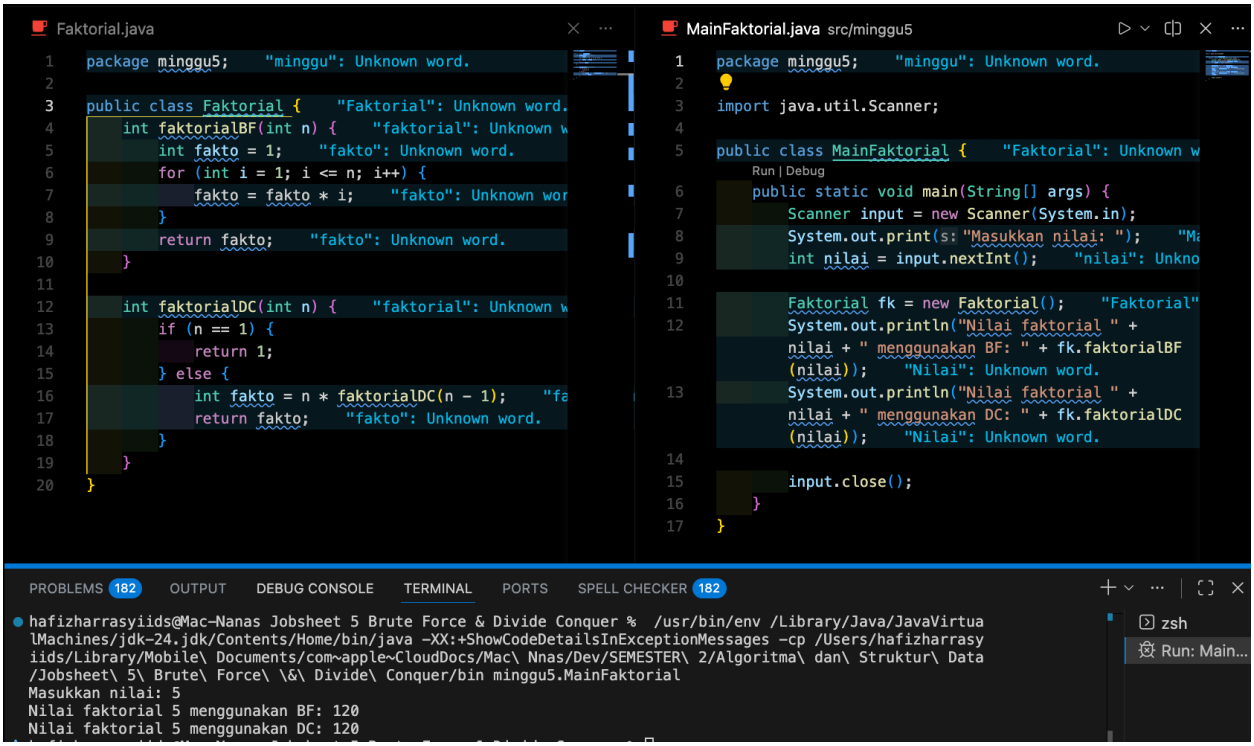
PROGRAM STUDI D4 SISTEM INFORMASI

BISNIS JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI

MALANG 2026

5.2.2 Verifikasi hasil percobaan



5.2.3 Pertanyaan

1. Pada base line Algoritma Divide Conquer untuk melakukan pencarian nilai faktorial, jelaskan perbedaan bagian kode pada penggunaan if dan else!
   Jawab :
   Bagian if (n==1) merupakan base case, yaitu kondisi penghentian rekursi. Ketika n bernilai 1, fungsi langsung mengembalikan nilai 1 tanpa melakukan pemanggilan rekursif lagi. Bagian else adalah bagian rekursif (recursive case) di mana fungsi memanggil dirinya sendiri dengan argumen n-1, kemudian mengalikan hasilnya dengan n. Proses ini terus berlanjut hingga mencapai base case.

2. Apakah memungkinkan perulangan pada method faktorialBF() diubah selain menggunakan for? Buktikan!
   Jawab :
   Ya, bisa. Berikut contoh menggunakan while:

```
int faktorialBF_while(int n) {
    int fakto = 1;
    int i = 1;
    while (i <= n) {
        fakto = fakto * i;
        i++;
    }
    return fakto;
}
```

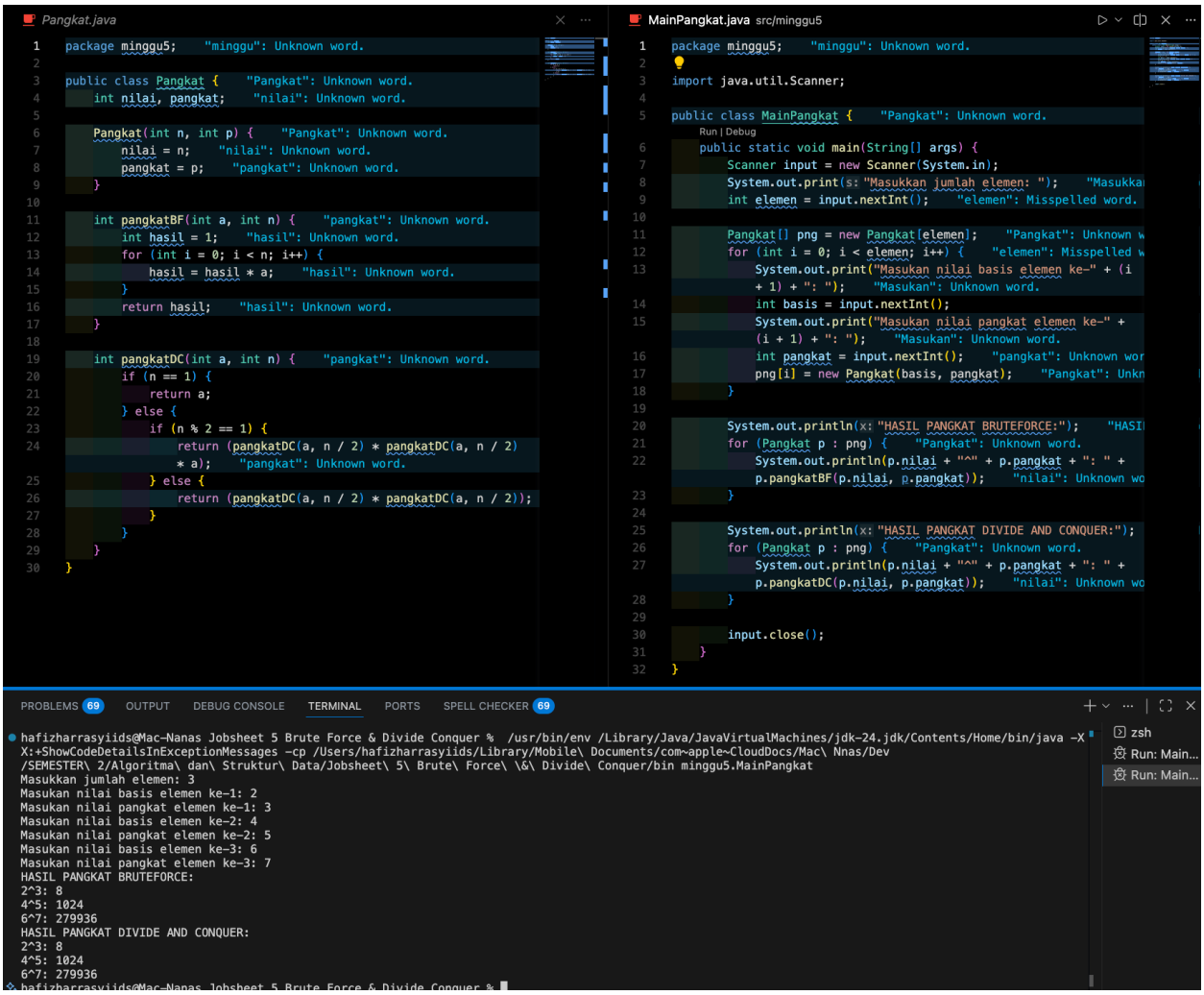
Dan menggunakan do-while:

```
    int faktorialBF_dowhile(int n) {
    int fakto = 1;
    int i = 1;
    do {
        fakto = fakto * i;
        i++;
    } while (i <= n);
    return fakto;
}
```

3. Jelaskan perbedaan antara fakto *= i; dan int fakto = n * faktorialDC(n-1); !
   Jawab :
   fakto *= i; adalah operasi assignment perkalian berulang secara iteratif. Nilai fakto diperbarui setiap iterasi dengan mengalikan nilai sebelumnya dengan i. Ini adalah pendekatan Brute Force yang berjalan secara linear dari 1 hingga n. int fakto = n * faktorialDC(n-1); adalah pemanggilan rekursif. Fungsi memanggil dirinya sendiri dengan n dikurangi 1, kemudian hasilnya dikalikan n. Ini adalah pendekatan Divide and Conquer di mana masalah dibagi menjadi sub-masalah yang lebih kecil.

4. Buat Kesimpulan tentang perbedaan cara kerja method faktorialBF() dan faktorialDC()!
   Jawab :
   faktorialBF() menggunakan perulangan for secara iteratif dari i=1 hingga i=n, mengalikan secara berurutan satu per satu. Cara kerjanya linier dan langsung. faktorialDC() menggunakan rekursi, memecah masalah n! menjadi n * (n-1)! terus hingga mencapai base case n=1. Keduanya menghasilkan hasil yang sama, namun DC menggunakan call stack rekursi sedangkan BF tidak.

5.3.2. Verifikasi Hasil Percobaan



Pangkat.java

```java
package minggu5;    "minggu": Unknown word.

public class Pangkat {    "Pangkat": Unknown word.
    int nilai, pangkat;    "nilai": Unknown word.

    Pangkat(int n, int p) {    "Pangkat": Unknown word.
        nilai = n;    "nilai": Unknown word.
        pangkat = p;    "pangkat": Unknown word.
    }

    int pangkatBF(int a, int n) {    "pangkat": Unknown word.
        int hasil = 1;    "hasil": Unknown word.
        for (int i = 0; i < n; i++) {
            hasil = hasil * a;    "hasil": Unknown word.
        }
        return hasil;    "hasil": Unknown word.
    }

    int pangkatDC(int a, int n) {    "pangkat": Unknown word.
        if (n == 1) {
            return a;
        } else {
            if (n % 2 == 1) {
                return (pangkatDC(a, n / 2) * pangkatDC(a, n / 2)
                * a);    "pangkat": Unknown word.
            } else {
                return (pangkatDC(a, n / 2) * pangkatDC(a, n / 2));
            }
        }
    }
}
```

MainPangkat.java src/minggu5

```java
package minggu5;    "minggu": Unknown word.

import java.util.Scanner;

public class MainPangkat {    "Pangkat": Unknown word.
    Run | Debug
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print(s: "Masukkan jumlah elemen: ");    "Masukka
        int elemen = input.nextInt();    "elemen": Misspelled word.

        Pangkat[] png = new Pangkat[elemen];    "Pangkat": Unknown w
        for (int i = 0; i < elemen; i++) {    "elemen": Misspelled w
            System.out.print("Masukan nilai basis elemen ke-" + (i
            + 1) + ": ");    "Masukan": Unknown word.
            int basis = input.nextInt();
            System.out.print("Masukan nilai pangkat elemen ke-" +
            (i + 1) + ": ");    "Masukan": Unknown word.
            int pangkat = input.nextInt();    "pangkat": Unknown wor
            png[i] = new Pangkat(basis, pangkat);    "Pangkat": Unkn
        }

        System.out.println(x: "HASIL PANGKAT BRUTEFORCE:");    "HASI
        for (Pangkat p : png) {    "Pangkat": Unknown word.
            System.out.println(p.nilai + "^" + p.pangkat + ": " +
            p.pangkatBF(p.nilai, p.pangkat));    "nilai": Unknown wo
        }

        System.out.println(x: "HASIL PANGKAT DIVIDE AND CONQUER:");
        for (Pangkat p : png) {    "Pangkat": Unknown word.
            System.out.println(p.nilai + "^" + p.pangkat + ": " +
            p.pangkatDC(p.nilai, p.pangkat));    "nilai": Unknown wo
        }

        input.close();
    }
}
```

PROBLEMS 69 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 69

```
hafizharrasyiids@Mac-Nanas Jobsheet 5 Brute Force & Divide Conquer %  /usr/bin/env /Library/Java/JavaVirtualMachines/jdk-24.jdk/Contents/Home/bin/java -X
X:+ShowCodeDetailsInExceptionMessages -cp /Users/hafizharrasyiids/Library/Mobile\ Documents/com~apple~CloudDocs/Mac\ Nnas/Dev
/SEMESTER\ 2/Algoritma\ dan\ Struktur\ Data/Jobsheet\ 5\ Brute\ Force\ \&\ Divide\ Conquer/bin minggu5.MainPangkat
Masukkan jumlah elemen: 3
Masukan nilai basis elemen ke-1: 2
Masukan nilai pangkat elemen ke-1: 3
Masukan nilai basis elemen ke-2: 4
Masukan nilai pangkat elemen ke-2: 5
Masukan nilai basis elemen ke-3: 6
Masukan nilai pangkat elemen ke-3: 7
HASIL PANGKAT BRUTEFORCE:
2^3: 8
4^5: 1024
6^7: 279936
HASIL PANGKAT DIVIDE AND CONQUER:
2^3: 8
4^5: 1024
6^7: 279936
hafizharrasyiids@Mac-Nanas Jobsheet 5 Brute Force & Divide Conquer %
```

zsh
Run: Main...
Run: Main...

5.3.3 Pertanyaan

1. Jelaskan mengenai perbedaan 2 method yang dibuat yaitu pangkatBF() dan pangkatDC()!
   Jawab :
   pangkatBF() menghitung a^n dengan cara mengalikan a sebanyak n kali secara iteratif menggunakan for loop (i dari 0 hingga n-1). Kompleksitas waktu O(n). pangkatDC() menggunakan Divide and Conquer dengan membagi pangkat n menjadi setengahnya. Jika n genap: a^n = a^(n/2) * a^(n/2). Jika n ganjil: a^n = a^(n/2) * a^(n/2) * a. Kompleksitas waktu O(log n), jauh lebih efisien untuk pangkat besar.

2. Apakah tahap combine sudah termasuk dalam kode tersebut?Tunjukkan!
   Jawab :
   Ya, tahap combine sudah ada. Pada pangkatDC(), combine dilakukan saat nilai rekursif digabungkan kembali:

```java
// Combine untuk pangkat genap:
return (pangkatDC(a, n/2) * pangkatDC(a, n/2));

// Combine untuk pangkat ganjil:
```

```
return (pangkatDC(a, n/2) * pangkatDC(a, n/2) * a);
```

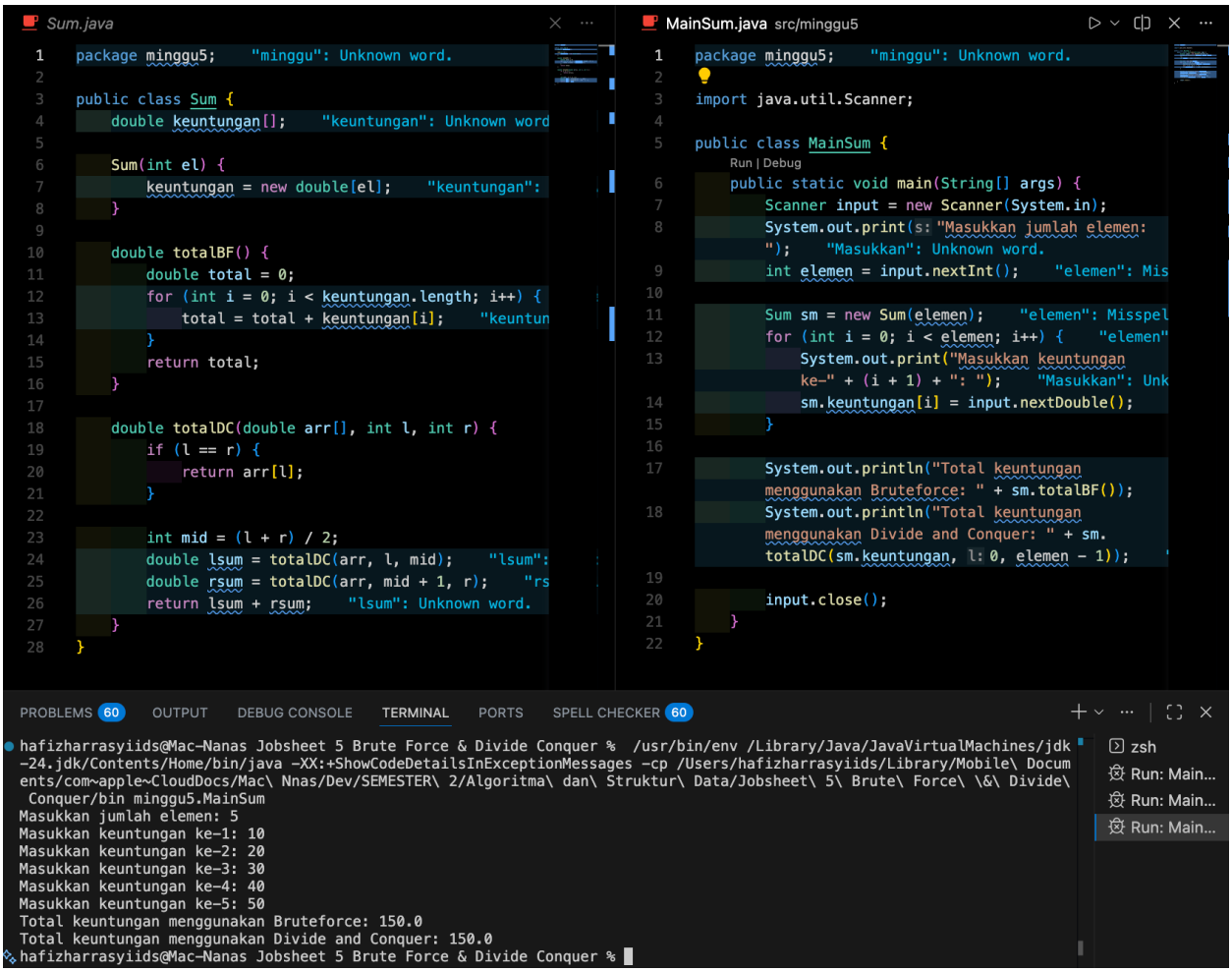
Hasil dari dua sub-masalah (pangkat setengah kiri dan kanan) digabungkan dengan operasi perkalian.

3. Pada method pangkatBF()terdapat parameter untuk melewatkan nilai yang akan dipangkatkan dan pangkat berapa, padahal di sisi lain di class Pangkat telah ada atribut nilai dan pangkat, apakah menurut Anda method tersebut tetap relevan untuk memiliki parameter? Apakah bisa jika method tersebut dibuat dengan tanpa parameter? Jika bisa, seperti apa method pangkatBF() yang tanpa parameter?
   Jawab :
   Method tetap relevan memiliki parameter karena memungkinkan penggunaan yang lebih fleksibel (dapat menerima nilai berbeda dari atribut objek). Namun bisa juga dibuat tanpa parameter dengan langsung menggunakan atribut kelas:

```
int pangkatBF() {
    int hasil = 1;
    for (int i = 0; i < pangkat; i++) {
        hasil = hasil * nilai;
    }
    return hasil;
}
```

4. Tarik tentang cara kerja method pangkatBF() dan pangkatDC()!
   Jawab :
   pangkatBF() melakukan perkalian berulang a sebanyak n kali secara linear, cocok untuk nilai pangkat kecil. pangkatDC() memecah masalah menjadi setengahnya setiap rekursi sehingga jumlah operasi jauh lebih sedikit (O(log n)). Untuk nilai pangkat yang besar, pangkatDC() jauh lebih efisien karena setiap langkah mengurangi masalah menjadi separuhnya.

5.4.2 Verifikasi Hasil Percobaan



Sum.java

```java
package minggu5;    "minggu": Unknown word.

public class Sum {
    double keuntungan[];    "keuntungan": Unknown word

    Sum(int el) {
        keuntungan = new double[el];    "keuntungan":
    }

    double totalBF() {
        double total = 0;
        for (int i = 0; i < keuntungan.length; i++) {
            total = total + keuntungan[i];    "keuntun
        }
        return total;
    }

    double totalDC(double arr[], int l, int r) {
        if (l == r) {
            return arr[l];
        }

        int mid = (l + r) / 2;
        double lsum = totalDC(arr, l, mid);    "lsum":
        double rsum = totalDC(arr, mid + 1, r);    "rs
        return lsum + rsum;    "lsum": Unknown word.
    }
}
```

MainSum.java src/minggu5

```java
package minggu5;    "minggu": Unknown word.

import java.util.Scanner;

public class MainSum {
    Run | Debug
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print(s: "Masukkan jumlah elemen:
        ");    "Masukkan": Unknown word.
        int elemen = input.nextInt();    "elemen": Mis

        Sum sm = new Sum(elemen);    "elemen": Misspel
        for (int i = 0; i < elemen; i++) {    "elemen"
            System.out.print("Masukkan keuntungan
            ke-" + (i + 1) + ": ");    "Masukkan": Unk
            sm.keuntungan[i] = input.nextDouble();
        }

        System.out.println("Total keuntungan
        menggunakan Bruteforce: " + sm.totalBF());
        System.out.println("Total keuntungan
        menggunakan Divide and Conquer: " + sm.
        totalDC(sm.keuntungan, l: 0, elemen - 1));

        input.close();
    }
}
```

PROBLEMS 60 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 60

```
hafizharrasyiids@Mac-Nanas Jobsheet 5 Brute Force & Divide Conquer %  /usr/bin/env /Library/Java/JavaVirtualMachines/jdk
-24.jdk/Contents/Home/bin/java -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/hafizharrasyiids/Library/Mobile\ Docum
ents/com~apple~CloudDocs/Mac\ Nnas/Dev/SEMESTER\ 2/Algoritma\ dan\ Struktur\ Data/Jobsheet\ 5\ Brute\ Force\ \&\ Divide\
 Conquer/bin minggu5.MainSum
Masukkan jumlah elemen: 5
Masukkan keuntungan ke-1: 10
Masukkan keuntungan ke-2: 20
Masukkan keuntungan ke-3: 30
Masukkan keuntungan ke-4: 40
Masukkan keuntungan ke-5: 50
Total keuntungan menggunakan Bruteforce: 150.0
Total keuntungan menggunakan Divide and Conquer: 150.0
hafizharrasyiids@Mac-Nanas Jobsheet 5 Brute Force & Divide Conquer %
```

zsh
Run: Main...
Run: Main...
Run: Main...

5.4.3 Pertanyaan

1. Kenapa dibutuhkan variable mid pada method TotalDC()?
   Jawab :
   Variabel mid dibutuhkan sebagai titik pembagi (pivot) array menjadi dua bagian. mid = (l + r) / 2 menentukan indeks tengah, sehingga bagian kiri mencakup indeks l hingga mid, dan bagian kanan mid+1 hingga r. Ini adalah inti dari proses Divide pada algoritma Divide and Conquer.

2. Untuk apakah statement di bawah ini dilakukan dalam TotalDC()?

```java
double lsum = totalDC(arr, l, mid);
double rsum = totalDC(arr, mid+1, r);
```

   Jawab :
   Statement double lsum = totalDC(arr, l, mid) adalah proses rekursi untuk menghitung total bagian kiri array (dari l hingga mid). Statement double rsum = totalDC(arr, mid+1, r) adalah proses rekursi untuk menghitung total bagian kanan array (dari mid+1 hingga r). Keduanya merupakan tahap Conquer, yaitu menyelesaikan sub-masalah secara rekursif.

3. Kenapa diperlukan penjumlahan hasil lsum dan rsum seperti di bawah ini?

```
return lsum+rsum;
```

Jawab :
return lsum + rsum adalah tahap Combine, yaitu menggabungkan hasil solusi dari dua sub-masalah (bagian kiri dan kanan) menjadi solusi keseluruhan. Penjumlahan kedua nilai ini menghasilkan total seluruh elemen array dari indeks l hingga r.

4. Apakah base case dari totalDC()?
Jawab :
Base case dari totalDC() adalah kondisi if (l == r), artinya ketika indeks kiri dan kanan sudah sama (hanya 1 elemen tersisa). Pada kondisi ini, fungsi langsung mengembalikan arr[l] karena tidak perlu dibagi lagi.

5. Tarik Kesimpulan tentang cara kerja totalDC()
Jawab :
totalDC() bekerja dengan tiga tahap: (1) Divide – membagi array menjadi dua bagian menggunakan mid, (2) Conquer – memanggil dirinya sendiri secara rekursif untuk masing-masing bagian hingga mencapai 1 elemen (base case), (3) Combine – menjumlahkan hasil dari kedua bagian. Proses ini terus berlanjut dari level bawah ke atas (bottom-up) hingga seluruh array terjumlahkan.

4.5 Latihan Praktikum

1. Sebuah kampus memiliki daftar nilai mahasiswa dengan data sesuai tabel di bawah ini

Nama	NIM	Tahun Masuk	Nilai UTS	Nilai UAS
Ahmad	220101001	2022	78	82
Budi	220101002	2022	85	88
Cindy	220101003	2021	90	87
Dian	220101004	2021	76	79
Eko	220101005	2023	92	95
Fajar	220101006	2020	88	85
Gina	220101007	2023	80	83
Hadi	220101008	2020	82	84

Tentukan:
   a) Nilau UTS tertinggi tertinggi menggunakan Divide and Conquer!
   b) Nilai UTS terendah menggunakan Divide and Conquer!
   c) Rata-rata nilai UAS dari semua mahasiswa menggunakan Brute Force!

MainLatihan.java src/minggu5

```java
package minggu5;    "minggu": Unknown word.

public class MainLatihan {    "Latihan": Unknown word.
    Run | Debug
    public static void main(String[] args) {
        NilaiMahasiswa[] mahasiswa = {    "Nilai": Unknown word.
            new NilaiMahasiswa(nama: "Ahmad", nim: "220101001",
            tahunMasuk: 2022, nilaiUTS: 78, nilaiUAS: 82),    "Nilai"
            new NilaiMahasiswa(nama: "Budi",  nim: "220101002",
            tahunMasuk: 2022, nilaiUTS: 85, nilaiUAS: 88),    "Nilai"
            new NilaiMahasiswa(nama: "Cindy", nim: "220101003",
            tahunMasuk: 2021, nilaiUTS: 90, nilaiUAS: 87),    "Nilai"
            new NilaiMahasiswa(nama: "Dian",  nim: "220101004",
            tahunMasuk: 2021, nilaiUTS: 76, nilaiUAS: 79),    "Nilai"
            new NilaiMahasiswa(nama: "Eko",   nim: "220101005",
            tahunMasuk: 2023, nilaiUTS: 92, nilaiUAS: 95),    "Nilai"
            new NilaiMahasiswa(nama: "Fajar", nim: "220101006",
            tahunMasuk: 2020, nilaiUTS: 88, nilaiUAS: 85),    "Nilai"
            new NilaiMahasiswa(nama: "Gina",  nim: "220101007",
            tahunMasuk: 2023, nilaiUTS: 80, nilaiUAS: 83),    "Nilai"
            new NilaiMahasiswa(nama: "Hadi",  nim: "220101008",
            tahunMasuk: 2020, nilaiUTS: 82, nilaiUAS: 84)    "Nilai":
        };

        int n = mahasiswa.length;    "mahasiswa": Unknown word.

        int maxUTS = NilaiMahasiswa.maxUTSDC(mahasiswa, l: 0, n -
        1);    "Nilai": Unknown word.
        System.out.println("a) Nilai UTS tertinggi (Divide and
        Conquer): " + maxUTS);    "Nilai": Unknown word.

        int minUTS = NilaiMahasiswa.minUTSDC(mahasiswa, l: 0, n -
        1);    "Nilai": Unknown word.
        System.out.println("b) Nilai UTS terendah (Divide and
        Conquer): " + minUTS);    "Nilai": Unknown word.

        double rataUAS = NilaiMahasiswa.rataUASBF(mahasiswa);    "Ni
        System.out.println("c) Rata-rata nilai UAS (Brute Force):
        " + rataUAS);    "nilai": Unknown word.
    }
}
```

NilaiMahasiswa.java

```java
package minggu5;    "minggu": Unknown word.

public class NilaiMahasiswa {    "Nilai": Unknown word.
    String nama;
    String nim;
    int tahunMasuk;    "tahun": Unknown word.
    int nilaiUTS;    "nilai": Unknown word.
    int nilaiUAS;    "nilai": Unknown word.

    NilaiMahasiswa(String nama, String nim, int tahunMasuk, int
    nilaiUTS, int nilaiUAS) {    "Nilai": Unknown word.
        this.nama = nama;
        this.nim = nim;
        this.tahunMasuk = tahunMasuk;    "tahun": Unknown word.
        this.nilaiUTS = nilaiUTS;    "nilai": Unknown word.
        this.nilaiUAS = nilaiUAS;    "nilai": Unknown word.
    }

    static int maxUTSDC(NilaiMahasiswa[] arr, int l, int r) {    "UT
        if (l == r) {
            return arr[l].nilaiUTS;    "nilai": Unknown word.
        }
        int mid = (l + r) / 2;
        int leftMax  = maxUTSDC(arr, l, mid);    "UTSDC": Unknown wo
        int rightMax = maxUTSDC(arr, mid + 1, r);    "UTSDC": Unknow
        return (leftMax > rightMax) ? leftMax : rightMax;
    }

    static int minUTSDC(NilaiMahasiswa[] arr, int l, int r) {    "UT
        if (l == r) {
            return arr[l].nilaiUTS;    "nilai": Unknown word.
        }
        int mid = (l + r) / 2;
        int leftMin  = minUTSDC(arr, l, mid);    "UTSDC": Unknown wo
        int rightMin = minUTSDC(arr, mid + 1, r);    "UTSDC": Unknow
        return (leftMin < rightMin) ? leftMin : rightMin;
    }

    static double rataUASBF(NilaiMahasiswa[] arr) {    "UASBF": Unkn
        double total = 0;
        for (int i = 0; i < arr.length; i++) {
            total += arr[i].nilaiUAS;    "nilai": Unknown word.
        }
        return total / arr.length;
    }
}
```

PROBLEMS 76 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 76

```
hafizharrasyiids@Mac-Nanas Jobsheet 5 Brute Force & Divide Conquer %  /usr/bin/env /Library/Java/JavaVirtualMachines/jdk-24.
jdk/Contents/Home/bin/java -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/hafizharrasyiids/Library/Mobile\ Documents/com
~apple~CloudDocs/Mac\ Nnas/Dev/SEMESTER\ 2/Algoritma\ dan\ Struktur\ Data/Jobsheet\ 5\ Brute\ Force\ \&\ Divide\ Conquer/bin
 minggu5.MainLatihan
a) Nilai UTS tertinggi (Divide and Conquer): 92
b) Nilai UTS terendah (Divide and Conquer): 76
c) Rata-rata nilai UAS (Brute Force): 85.375
hafizharrasyiids@Mac-Nanas Jobsheet 5 Brute Force & Divide Conquer %
```

zsh
Run: Main...

LINK GITHUB : https://github.com/nNasq/Jobsheet-5-Brute-Force---Divide-Conquer